



Curso: Inteligencia Artificial (CC421)

Sección 1: Preguntas de opción múltiple [1 punto c/u → 4 puntos]

Instrucciones: Seleccione la única respuesta correcta en cada caso.

1. ¿Cuál de estos entornos corresponde mejor a un sistema de recomendación de películas en línea?
 - a) Estocástico, parcialmente observable, secuencial.
 - b) Determinístico, completamente observable, episódico.
 - ☒ c) Estocástico, completamente observable, continuo.
 - d) Determinístico, parcialmente observable, secuencial.
 - e) Estocástico, parcialmente observable, sinérgico.
2. El enfoque de **Actuar humanamente** en IA está directamente asociado a:
 - a) Modelado de procesos cerebrales.
 - ☒ b) Prueba de Turing.
 - c) Razonamiento basado en lógica formal.
 - d) Maximización de la utilidad esperada.
 - e) Sistemas expertos.
3. Un entorno estocástico, parcialmente observable y secuencial corresponde mejor a:
 - a) Un robot industrial ensamblando piezas con sensores completos.
 - ☒ b) Un chatbot conversacional que mide el tono de voz del usuario.
 - c) Un juego de ajedrez en línea con información completa.
 - d) Un sistema de recomendación de películas.
 - e) Un algoritmo de clasificación supervisada con datos limpios.
4. En A*, la función de evaluación $f(n)$ se compone de:
 - a) $f(n) = g(n)$ si la heurística es admisible
 - b) $f(n) = h(n)$ para minimizar la profundidad de búsqueda.
 - ☒ c) $f(n) = g(n) + h(n)$ combinando costo acumulado y estimado.
 - d) $f(n) = \max\{g(n), h(n)\}$ para garantizar consistencia.
 - e) $f(n) = g(n) - h(n)$ para priorizar rutas más lejanas.

Sección 2: Verdadero/Falso [1 punto c/u → 4 puntos]

Instrucciones: Cada pregunta tiene tres proposiciones. Indique la combinación (VVV, VVF, VFF, etc.) y justifique brevemente cada una (1 - 2 líneas).

1.
 - ☐ Una heurística consistente siempre cumple $h(n) \leq c(n, a, n') + h(n')$.
 - ☒ La búsqueda greedy ¿mejor primero? garantiza optimalidad si la heurística es admisible.
 - ☒ A* con heurística admisible y consistente es completa y óptima.
2.
 - ☒ Un entorno desconocido requiere un agente autónomo.
 - ☒ Los entornos estáticos no cambian durante la actuación del agente.
 - ☒ Un entorno multiagente es inherentemente competitivo o colaborativo.
3.
 - ☐ La búsqueda en profundización iterativa evita el problema de memoria del DFS.
 - ☒ La búsqueda bidireccional siempre es más eficiente que la unidireccional.
 - ☒ Una heurística consistente (monótona) implica siempre admisibilidad.
4.
 - ☒ Un agente reflexivo simple carece de memoria y no almacena estados previos.
 - ☒ Un agente de utilidad elige acciones solo según reglas rígidas, sin considerar probabilidades.
 - ☒ La búsqueda en amplitud garantiza hallar la solución de menor profundidad, aun si es costosa.

Sección 3: Pseudocódigo [2 puntos c/u → 4 puntos]

Instrucciones: Redacte el pseudocódigo solicitado e incluya un comentario breve (1-2 líneas).

1. Se desea resolver un problema de ruta en una cuadrícula (grid) donde un agente debe moverse desde un punto inicial (S) hasta una meta (G), evitando obstáculos (X). El costo de movimiento horizontal es 1, y vertical es 1, y diagonal es raíz de 2. La heurística a usar es la distancia de Manhattan. Elabore el algoritmo pseudocódigo de A* para que:
 - Priorice nodos con menor $f(n) = g(n) + h(n)$ donde $g(n)$ es el costo acumulado y $h(n)$ la heurística.
 - Evite ciclos verificando nodos ya visitados.
 - Retorne la ruta óptima o *No hay solución*.
2. Implemente en pseudocódigo el algoritmo **Uniform-Cost Search** para un grafo genérico, indicando cómo maneja la frontera y la prioridad.

Sección 4: Pregunta de desarrollo [2 puntos]

Problema: Resuelva el 8-puzzle con el estado inicial $[[2, 8, 3], [1, 6, 4], [7, 0, 5]]$ y meta $[[1, 2, 3], [8, 0, 4], [7, 6, 5]]$ usando A*.

▪ Heurística: Distancia Manhattan.

▪ Pasos:

1. Calcular heurística para el estado inicial: suma de distancias de cada ficha a su posición meta.
2. Expandir nodos priorizando el menor costo acumulado + heurística.
3. Mostrar el árbol hasta alcanzar la meta.

Sección 3: Pseudocódigo [3 puntos c/u $\rightarrow$ 6 puntos]

Instrucciones: Redacte el pseudocódigo solicitado e incluya un comentario breve (1 o 2 líneas).

1. (CSP - Backtracking con Heurística MRV) Escriba el pseudocódigo para una función *BacktrackingSearch_CSP(csp)* que implemente el algoritmo de backtracking para resolver un Problema de Satisfacción de Restricciones (CSP). Su implementación debe incluir la selección de la siguiente variable a asignar utilizando la heurística de **Mínimo Valor Restante (MRV)**. Asuma que tiene acceso a funciones como *SelectUnassignedVariable_MRV(csp, assignment)*, *OrderDomainValues(var, assignment, csp)*, *isComplete(assignment)*, y *isConsistent(var, value, assignment, csp)*. Explique brevemente la lógica principal y el papel de la heurística MRV.
2. (Búsqueda Adversaria - Función Max-Value con Poda Alfa-Beta) Escriba el pseudocódigo para la función *MaxValue(state, alpha, beta)* que forma parte del algoritmo de poda Alfa-Beta. Esta función calcula el mejor valor posible para el jugador MAX desde un estado dado, considerando los límites alpha (mejor valor encontrado hasta ahora para MAX en la ruta) y beta (mejor valor encontrado hasta ahora para MIN en la ruta). Asuma la existencia de funciones como *isTerminal(state)*, *Utility(state)*, *Actions(state)* y *Result(state, action)*, y la función *MinValue*. Explique cómo se utilizan alpha y beta para realizar la poda.

Sección 4: Pregunta de desarrollo [4 puntos]

(Problemas de Satisfacción de Restricciones - CSP) Considere el siguiente problema de coloreado de mapa con 4 regiones (WA, NT, SA, Q) y 3 colores disponibles (Rojo, Verde, Azul). Las restricciones de adyacencia son las siguientes:

- WA es adyacente a NT y SA.
- NT es adyacente a WA, SA y Q.
- SA es adyacente a WA, NT y Q.
- Q es adyacente a NT y SA.

Tarea: Encuentre una asignación de colores válida para todas las regiones utilizando el algoritmo de **Backtracking Search** combinado con la heurística **Minimum Remaining Values (MRV)** para la selección de variables y **Forward Checking** para la propagación de restricciones.

Muestre los siguientes pasos en su solución:

1. Defina formalmente las Variables (V), los Dominios (D) iniciales y las Restricciones (C) del CSP.
2. Describa el proceso de asignación paso a paso:
 - Indique qué variable se selecciona en cada paso según MRV (si hay empate, elija en orden alfabético: NT, Q, SA, WA).
 - Indique qué valor se asigna (si hay opciones, elija en orden: Rojo, Verde, Azul).
 - Muestre el estado de los dominios de las variables no asignadas después de aplicar Forward Checking tras cada asignación.
3. Indique la asignación final completa encontrada.

Resolver el problema detallando los pasos y mostrando el proceso de asignación y chequeo de restricciones.